SIMATS ENGINEERING
ASSESSMENT TOOL

COURSE NAME: Theory of Computation COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO2: De**ign Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)**

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:20
Duration : 1 Hour

Scenario Based

Code of Conduct:

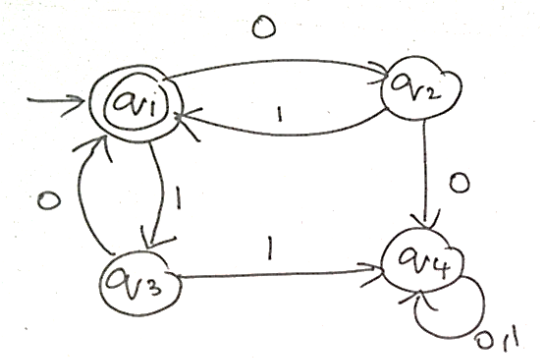
I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING
ASSESSMENT TOOL

1.Given a DFA used for validating a specific PIN pattern bbbc, bca ,c, ca, caaaaaa minimize the DFA and explain how state reduction improves performance and efficiency in embedded systems

2.Demonstrate how a given finite automaton can be converted into a regular expression



3.Consider the following grammar

S->aSc|T|U|e

T-> bTc|e

U->aUb|e

A} What is the language defined by the grammar?

B)Give the derivation tree for the word aaabbc

C Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

4.I)Show that if L1 and L2 are regular then (L1+l2) also regular

Ii)Is the language { a^p| p is prime} regular? Justify

SIMATS ENGINEERING
ASSESSMENT TOOL

:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Theory of Computation (CSA13) - CO2 Scenario Based

Worked solutions / study notes

Study material only. Rewrite in your own words and complete your own Code of Conduct declaration on the official assessment sheet.

Q1. Minimizing the PIN DFA {bbbc, bca, c, ca, caaaaaa}

No automaton is drawn, so the DFA is built as a trie of the five PINs. The raw trie has 14 states plus a dead state.

Merges. (1) The three accepting states reached after bbbc, bca and caaaaaa have no outgoing moves, so they are equivalent: merge into F. (2) The non-final states after 'bc' and after 'caaaaa' both move only on a to F, so they are equivalent: merge into X. Result: 11 states plus a dead state (missing transitions go to the dead state).

State	a	b	c
→ s	-	b1	C0
b1	-	b2	X
b2	-	b3	-
b3	-	-	F
X	F	-	-
* C0 (c)	C1	-	-
* C1 (ca)	C2	-	-
C2	C3	-	-
C3	C4	-	-
C4	X	-	-
* F	-	-	-

Check: bbbc = s, b1, b2, b3, F. bca = s, b1, X, F. caaaaaa = C0, C1, C2, C3, C4, X, F.

How state reduction helps embedded systems:

- Smaller transition table, so less ROM/RAM on resource-limited microcontrollers.
- Fewer state bits and less logic in a hardware implementation.
- Lower power consumption and fewer lookups per input symbol.
- Easier to test and verify; the minimal DFA is unique, so designs can be compared for equivalence.

Q2. Finite automaton to regular expression

From the diagram: q1 is the start and only final state. q1 goes to q2 on 0 and to q3 on 1; q2 goes to q1 on 1 and to q4 on 0; q3 goes to q1 on 0 and to q4 on 1; q4 loops on 0,1.

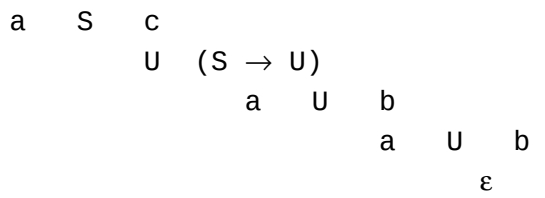
State elimination:

1. q4 is a dead state (cannot reach q1): remove it.
2. Eliminate q2: path q1 to q2 to q1 gives a loop 01 on q1.
3. Eliminate q3: path q1 to q3 to q1 gives a loop 10 on q1.
4. q1 is start and final with loops 01 and 10.

Answer: (01 + 10)*

Q3. Grammar S → aSc | T | U | ε, T → bTc | ε, U → aUb | ε

(A) T generates $b^n c^n$, U generates $a^n b^n$; each S → aSc adds an a and a c around the middle. L = { $a^i b^j c^k$: i = j + k or k = i + j }.
(B) aaabbc has a = b + c, so use the U branch: S ⇒ aSc ⇒ aUc ⇒ aaUbc ⇒ aaaUbbc ⇒ aaabbc.



(C) Ambiguous. The empty string has three derivations: S ⇒ ε, S ⇒ T ⇒ ε, S ⇒ U ⇒ ε. 'ac' also has two parse trees (via T and via U).

Q4

(i) Union is regular. Take NFAs M1, M2 for L1, L2. Add a new start state with ε-moves to both start states; final states are the union of both final sets. A string is accepted iff M1 or M2 accepts it, so this ε-NFA accepts L1 ∪ L2, which is therefore regular.
(ii) $\{a^p$: p prime$\}$ is not regular. Assume regular with pumping length n; pick prime p ≥ n, w = a^p = xyz, |y| = m ≥ 1, |xy| ≤ n. Pump y p+1 times: length = p + pm = p(1+m). Since p ≥ 2 and 1+m ≥ 2, this is composite, so not in the language. Contradiction.